Green Serverless Computing for Resource-Efficient AI Training

Executive Summary

Green serverless computing aims to combine the elasticity and operational simplicity of serverless platforms with explicit optimization for energy efficiency and carbon footprint, particularly for AI training workloads that are extremely compute- and energy-intensive. Recent research projects (for example, VinUni's "Green Serverless Computing for Resource-Efficient AI Training") target new architectures and schedulers that minimize idle capacity, cold-start overheads, and carbon intensity while preserving performance. Work such as LoAPE and LowCarb shows that load-aware, power-elastic and carbon-aware scheduling can yield near-optimal trade-offs between latency, throughput, and environmental impact in serverless platforms.[1][2][3][4][5][6][7]

This report provides a deep dive along two axes:

- Mathematical models describing energy, cold start overhead, utilization, and carbon-aware scheduling for serverless AI training.
- Concrete technical mechanisms (at code, runtime, and scheduler level) to reduce energy use and emissions: cold-start mitigation, load-aware scaling, GPU sharing, data movement minimization, and carbon-aware placement.[8][6][1]

The focus is primarily on function-as-a-service (FaaS) and serverless GPU workers used for data preprocessing, micro-batch training, and auxiliary training tasks, rather than monolithic, tightly coupled multi-GPU training jobs.

Background: AI Energy Use and Serverless

Energy and Carbon in AI Training

AI training, especially for large-scale deep learning models, is a major driver of data center energy consumption and associated carbon emissions. Estimates suggest that demand from AI-related computing could at least double data center energy consumption by 2030 if not carefully optimized. Specialized accelerators such as GPUs and NPUs provide large gains in performance per watt compared to CPUs, but their sheer scale of deployment still leads to significant environmental impact.[5][9]

Serverless computing is attractive for AI because it provides:

- Fine-grained resource allocation (per function or micro-batch).
- Automatic scaling with workload, reducing the need for always-on overprovisioned clusters.
- Pay-per-use pricing, which naturally correlates with energy usage in many cloud offerings.[8][10]

However, serverless introduces challenges such as cold starts, function fragmentation, and potential overheads from container lifecycle and data transfers, all of which can erode energy efficiency if not managed carefully.[11][12][8]

Green Serverless Computing

"Green serverless computing" refers to serverless designs that explicitly integrate energy- and carbon-related objectives into platform and application-level decisions. This typically involves:[8][10]

- Minimizing idle capacity and overprovisioning while satisfying performance SLAs.
- Reducing the energy and latency overhead of cold starts and container lifecycle operations.
- Exploiting renewable energy availability and carbon-aware scheduling across regions and time periods.[6][13][8]

Projects like VinUni's "Green Serverless Computing for Resource-Efficient AI Training" focus on designing and evaluating serverless architectures specifically tailored to AI training workloads, emphasizing energy measurements, scheduling strategies, and prototype implementations on public clouds.[2][7]

Mathematical Foundations

Energy Model for Serverless Functions

Consider a serverless function handling AI-related work (e.g., a data preprocessing task or a micro-batch training step). For each invocation, total energy can be decomposed into:

$$E_{total} = E_{work} + E_{overhead}$$

where E_{work} is the energy for useful computation, and $E_{overhead}$ covers cold starts, container lifecycle, and idle intervals between invocations.[8][12]

If a task requires C floating point operations (FLOPs) on hardware with energy efficiency η FLOPs per joule, then the ideal work energy is:

$$E_{work} = \frac{C}{\eta}$$

For AI accelerators, η can reach up to on the order of 10^{12} FLOPs/W for state-of-the-art NPUs, significantly higher than general-purpose CPUs. The challenge in serverless is to keep $E_{overhead}$ small compared with E_{work} for each invocation.[5]

Cold Start Overhead

A cold start occurs when no warm execution environment is available and the platform must allocate resources, initialize a runtime, and load function code and dependencies. Let:[11][12]

- t_{cs}: cold start duration.
- P_{cs}: approximate average power during initialization.

Energy overhead per cold start is:

$$E_{cs} = P_{cs} \cdot t_{cs}$$

For N function invocations, if a fraction p of them experience cold starts, the total cold-start energy is:

$$E_{cs,total} = pNP_{cs}t_{cs}$$

The relative overhead compared to useful work is:

$$\frac{E_{cs,total}}{E_{work,total}} = \frac{pP_{cs}t_{cs}}{E_{work,avg}}$$

where $E_{work,avg}$ is the average work energy per invocation. For short-running or light-weight functions (low $E_{work,avg}$), cold starts can dominate energy usage unless mitigated via warm pools, provisioning, or batching.[8][11]

Warm Pool vs. Cold Start Trade-off

Platforms can maintain a pool of pre-initialized function instances to reduce cold starts. Each warm instance consumes idle power P_{idle} during time periods without work. Over a time interval T, the energy cost of keeping k instances warm is:

$$E_{warm} = kP_{idle}T$$

Suppose the average arrival rate is λ invocations per second, and each invocation would incur cold start energy E_{cs} if no warm instance is available. The expected cold start energy without a warm pool is roughly:

$$E_{cs,total} \approx \lambda T E_{cs}$$

A simple break-even condition (energy-only) for choosing a warm pool is:

$$E_{warm} < E_{cs,total} \implies kP_{idle}T < \lambda T E_{cs} \implies \lambda > \frac{kP_{idle}}{E_{cs}}$$

Thus, for sufficiently high arrival rates, keeping some instances warm is both faster and more energy-efficient; for low rates, relying on cold starts and terminating idle instances is greener.[8][12]

Queueing and Utilization

Serverless platforms can be approximated by queueing systems, for example an $M/M/k$ model with arrival rate λ, service rate μ per instance, and k concurrent instances.[3][4]

Utilization per instance is:

$$\rho = \frac{\lambda}{k\mu}$$

Energy has two components:

- Dynamic energy E_{dyn}, proportional to active compute time.
- Static energy E_{static}, proportional to number of active instances and their idle power.

If E_{dyn} per unit service time is e_{dyn} and E_{static} per instance per unit time is e_{static}, then over T:

$$E_{total} = e_{dyn} \cdot \lambda T/\mu + e_{static} \cdot kT$$

The green optimization problem is to choose k and, more generally, time-varying $k(t)$ and placement decisions to minimize E_{total} (or carbon-weighted energy) subject to constraints on latency (e.g., tail response time) and training throughput.[6][3]

Carbon-Aware Scheduling

Energy from different data centers or regions has different carbon intensity $g_r(t)$ (grams of CO_2 per kWh), varying by location r and time t. Carbon-aware schedulers aim to minimize total emissions:[8][6][13]

$$C = \sum_r \int g_r(t) P_r(t)\, dt$$

where $P_r(t)$ is power draw in region r at time t.

LowCarb is an example of a reinforcement learning-based scheduler that learns to place and keep-alive serverless functions across regions and time, optimizing a reward function that trades off response time and carbon footprint. It achieves performance and carbon metrics within about fifteen percent of an oracle optimal solution, demonstrating that such multi-objective scheduling is feasible at runtime.[6]

For AI training, many workloads are partially deferrable (e.g., hyperparameter sweeps, offline re-training), making them good candidates for time-shifting to periods of low grid carbon intensity.

Role of Serverless in AI Training

Decomposition of Training Workloads

Traditional deep learning training uses monolithic clusters of GPUs or TPUs managed by frameworks such as PyTorch Distributed or Horovod, often on dedicated VMs or bare metal. Serverless models are better suited for tasks that can be decomposed into relatively independent, stateless units:[5][9]

- Data preprocessing and augmentation.
- Feature extraction and embedding computation.
- Micro-batch training steps for smaller models or for certain phases like hyperparameter tuning.
- Auxiliary tasks, such as gradient aggregation for parameter servers.

Green serverless computing targets these components to:

- Exploit fine-grained auto-scaling according to data arrival or job queue lengths.
- Power down resources aggressively between bursts.
- Place computation where energy is greenest or cheapest.[1][2][8]

Benefits and Limitations

Key benefits include:

- Reduced idle capacity compared to always-on training clusters that must be sized for peak load.[8][10]
- Simplified management and potential alignment of billing with energy usage.
- Natural integration with event-driven data sources (e.g., streaming logs, IoT telemetry) that feed training pipelines.

Limitations and challenges:

- Cold starts and limited control over hardware can conflict with strict throughput requirements for large training runs.
- GPU-accelerated serverless offerings are still emerging and may have constraints on runtime duration, memory, or networking needed for distributed training.[9]
- Function granularity that is too fine can exacerbate overheads from initialization, data transfer, and scheduling.

Cold Starts: Performance and Energy

Lifecycle of a Cold Start

In platforms such as AWS Lambda, a cold start involves container provisioning, runtime initialization, function code loading, and dependency resolution. To conserve resources, idle environments are shut down, so future invocations must re-incur these costs. Similar patterns exist in open-source platforms like OpenWhisk and Knative.[12]

Cold starts impact both latency and energy:

- Latency: Added milliseconds to seconds before work begins, problematic for latency-sensitive tasks.
- Energy: Extra CPU and I/O cycles for initialization that provide no direct training progress.

Mitigation Strategies and Their Trade-offs

Common mitigation strategies include:

- Warm pools and provisioned concurrency: Keeping a number of pre-initialized instances available, reducing cold starts but increasing static energy.[8][12]
- Snapshotting and restore: Capturing initialized runtimes (e.g., after loading frameworks like PyTorch) and restoring from snapshots instead of cold booting.
- Lightweight runtimes: Using lean languages and minimal dependencies to reduce t_{cs} and, therefore, E_{cs}.[14][11]

Energy-wise, optimal strategies balance:

- The energy saved by avoiding cold starts.
- The extra energy from keeping instances warm.

As shown in the break-even inequality earlier, this depends strongly on arrival rates and function duration. Research on dynamic caching and routing in OpenWhisk has demonstrated that intelligently steering requests to recently used containers can significantly reduce both cold start frequency and tail latency.[14]

Load-Aware and Power-Elastic Platforms

LoAPE: A Green Serverless Platform

LoAPE (Load-Aware and Power-Elastic Platform for Green Serverless Computing) proposes architectural and algorithmic changes to serverless platforms to improve energy efficiency. While details are specific to the proposed system, key ideas include:[3][4]

- Dynamically adapting the number of active nodes and containers to current load.
- Power-elastic behavior, where server nodes can operate in different power states, scaling voltage and frequency according to utilization.
- Integrating these mechanisms into the serverless control plane so that function scheduling and scaling policies are aware of power and energy metrics, not just CPU and memory usage.

Mathematically, this corresponds to solving an optimization problem over time-varying decisions $k(t)$ (active instances) and power states $s(t)$ (e.g., DVFS levels) to minimize time-integrated power subject to bounds on response-time metrics.

Carbon-Aware Scheduling with LowCarb

LowCarb addresses a complementary challenge: scheduling serverless functions to minimize carbon footprint while meeting performance objectives. It models scheduling as a reinforcement learning (RL) problem, where actions include choosing execution regions, deciding whether to keep a function warm, and scaling capacity.[6]

Key aspects:

- The RL agent observes function characteristics, workload patterns, and current grid carbon intensity per region.
- The reward function penalizes both high carbon intensity and high service time, implicitly encoding a trade-off.[6]
- Experiments show the scheme achieves results within about fifteen percent of an optimal oracle across several metrics, and can augment existing serverless schedulers without breaking the abstraction for application developers.[6]

For AI training workflows, LowCarb-like schedulers can:

- Place bursty training-related tasks in regions with low carbon intensity at that time.
- Shift deferrable workloads (e.g., retraining, hyperparameter sweeps) into low-carbon windows.
- Decide which function types justify being kept warm versus allowed to cold start, depending on both expected latency impact and carbon trade-offs.

Green Design Patterns for Serverless AI Training

Naive Approach

A naive architecture for AI training with serverless might:

- Use serverless functions for every fine-grained step (e.g., per-sample or per-mini-batch preprocessing and gradient update).
- Rely entirely on default autoscaling with minimal configuration.
- Ignore locality, carbon intensity, and warm/cold dynamics.

This leads to:

- High cold-start frequency due to many small, infrequent function invocations.
- Repeated initialization of large ML frameworks (e.g., importing PyTorch at every cold start).
- Increased data transfer energy because each invocation separately fetches model parameters and data shards.[8][11]

Better: Batching and Function Fusion

An improved design uses batching and function fusion:

- Batch multiple samples or micro-batches into a single function invocation, so each cold start amortizes over more useful work.
- Fuse sequential steps (e.g., data preprocessing and local gradient computation) into a single function to avoid intermediate serialization and container hops.

The effective E_{cs} per training example decreases roughly as $1/B$ where B is the batch size processed per invocation, assuming cold-start cost is independent of B. At the same time, there is a trade-off with latency and memory usage.

Optimal Direction: Coordinated, Carbon-Aware Serverless Pipelines

An optimal or near-optimal green serverless AI training architecture integrates several techniques:

1. **Hierarchical decomposition of the training job**
    - Long-lived, tightly coupled GPU jobs (for the core model state) run on energy-efficient clusters, possibly in regions with better renewable penetration.[5][9]
    - Serverless functions handle elastic components: data ingestion, feature extraction, augmentation, and evaluation.
2. **Load-aware scaling and power elasticity**
    - Serverless platform adopts LoAPE-like features: nodes and instances are scaled and power-managed according to observed load and function characteristics.[3][4]
3. **Carbon-aware placement and scheduling**
    - Use a LowCarb-like RL scheduler to decide where and when to execute serverless tasks, co-optimizing for latency and carbon.[6][13]
4. **Aggressive cold-start mitigation with energy-aware thresholds**
    - Maintain warm pools for high-QPS or latency-critical functions (e.g., online feature extraction for real-time learning), sized using break-even conditions that factor in both energy and latency constraints.
    - Allow low-QPS or non-critical tasks to incur cold starts, scheduled into low-carbon windows.
5. **Data movement minimization**
    - Co-locate data preprocessing functions with storage to reduce network energy.
    - Cache frequently used datasets in regional storage near green energy sources when possible.[8]
6. **Hardware-aware code optimization**
    - Use tools like "icebreaker"-style analyzers to reduce function package size and dependency bloat, lowering t_{cs} and cold-start impact.[15]
    - Exploit accelerator-friendly implementations (vectorization, mixed precision) to reduce C FLOPs for a

given training task.[5]

Case Study Elements from Emerging Work

Green Serverless at VinUni

The VinUni Green Serverless Computing project explicitly aims to design and evaluate a serverless architecture for resource-efficient AI training, including:

- Designing serverless-based training pipelines that reduce resource waste compared with traditional fixed clusters.
- Implementing prototypes on public cloud platforms such as AWS Lambda or Azure Functions.
- Measuring performance, scalability, and energy efficiency under real and benchmark workloads.[1][2][7]

Although detailed results are still under development, this type of work exemplifies the shift from purely performance-oriented serverless designs towards energy- and carbon-aware architectures.

Energy-Efficient Serverless in Data Centers

Broader analyses of serverless computing in data centers emphasize that serverless can reduce idle capacity and improve resource utilization, but its net energy impact depends on container reuse, cold-start frequency, data-transfer overhead, and efficiency of the underlying infrastructure.[8][10]

Energy-saving aspects include:

- Fine-grained scaling to current demand, avoiding overprovisioning for peak.
- Granular resource allocation per function, reducing waste.
- Improved capacity planning based on real-time analytics.[8]

Challenges include:

- Overheads from frequent container start/stop cycles.
- Potential increases in data movement.

- Need for developer practices that minimize function footprint and execution time.[10][8]

Practical Guidelines for Practitioners

For practitioners designing green serverless systems for AI training, several concrete guidelines emerge from the literature and industrial practice:

1. **Profile and quantify energy and latency**
    - Instrument functions to estimate energy usage via hardware counters, cloud energy APIs, or empirical models.
    - Measure cold-start frequency and overhead; target high-impact functions for optimization first.[8][12]
2. **Optimize function packages and runtimes**
    - Remove unused dependencies, avoid monolithic libraries, and use lighter languages/runtimes where possible.
    - Employ tools that analyze function size and import graph to predict and reduce cold-start durations.[14][15]
3. **Batch work and fuse functions**
    - Combine small tasks into larger invocations when consistent with latency and memory constraints.
    - Fuse sequential steps to reduce container hops and data transfer.
4. **Tune warm pools with energy-aware policies**
    - Use simple models (like the break-even inequality above) to decide which functions merit warm instances.
    - Explore platform features such as provisioned concurrency with autoscaling policies based on utilization.
5. **Adopt carbon-aware scheduling**
    - Integrate carbon-intensity data from grid APIs and schedule deferrable training tasks into greener periods and regions.
    - Consider RL-based or heuristic multi-objective schedulers similar to LowCarb.[6][13]
6. **Choose energy-efficient hardware and regions**
    - Prefer accelerators and data centers with higher performance-per-watt and better renewable-energy integration.[5][9]

Conclusion

Green serverless computing for resource-efficient AI training sits at the intersection of serverless platform design, AI systems, and sustainable computing. Research and early deployments show that, with appropriate mathematical modeling of energy and carbon costs, and with technical mechanisms such as load-aware scaling, power elasticity, cold-start mitigation, and carbon-aware scheduling, serverless platforms can support AI training workloads with significantly improved energy and carbon efficiency compared to naive designs.[1][2][3][6]

However, realizing these gains in practice requires end-to-end consideration of workload decomposition, data locality, hardware characteristics, and developer practices. As serverless GPU offerings and energy-reporting capabilities mature, green serverless AI training is likely to become a mainstream architectural option for organizations seeking to align AI capabilities with sustainability goals.[8][5][10]

References

1. Green Serverless Computing for Resource-Efficient Model Training - The Green Serverless Computing project demonstrates how architectural innovation, when paired with r...
2. Green Serverless Computing for Resource-Efficient AI Training - This research aims to design and evaluate a green serverless computing architecture that supports re...
3. LoAPE: A Load-Aware and Power-Elastic Platform for Green ... - LoAPE: A Load-Aware and Power-Elastic Platform for Green Serverless Computing. Abstract: Energy effi...
4. LoAPE: A Load-Aware and Power-Elastic Platform for Green ... - Energy efficiency in serverless computing has remained under ... LoAPE: A Load-Aware and Power-Elast...
5. The Growing Importance of Energy Efficiency in AI Processing | Be - Energy-efficient AI can extend battery life, minimize downtime, and improve reliability for these sy...
6. LowCarb: Carbon-Aware Scheduling of Serverless Functions - Publications - Faasrank: Learning to schedule functions in

serverless platforms. In. 2021 IEEE International Confer...

7. Research Assistant – Green Serverless Computing - Seeking motivated and detail-oriented Research Assistants to contribute to our cutting-edge project ...
8. Is Serverless Computing in Data Centers Energy Efficient? - Green serverless computing involves using renewable energy sources in data centers. Cloud providers ...
9. Flexible, secure, sustainable: top inference for your AI applications - Yorizon offers highly scalable, energy-efficient cloud infrastructures, specifically optimized for m...
10. The Impact of Serverless Computing on Modern Data Centers - Advancing towards green serverless computing involves implementing energy-efficient practices within...
11. Conquering Serverless Cold Starts: Strategies for Optimal ... - A cold start refers to the delay a serverless function experiences when it's invoked for the first t...
12. Understanding and Remediating Cold Starts: An AWS Lambda ... - A cold start happens when there isn't an existing execution environment available and a new one must...
13. Carbon-Aware Cloud Workloads: Scheduling for a Greener Grid - This involves training machine learning models on historical carbon intensity data to predict future...
14. Optimizing Cold Start Performance in Serverless Computing ... - This work proposes an Apache Open Whisk-integrated dynamic caching and request routing system to mit...
15. Optimizing AWS Lambda Cold Start Duration with Our Serverless ... - Serverless Icebreaker is specifically engineered to optimize AWS Lambda's cold start duration. After...